Alawyer API

The Alawyer API lets you submit a legal question and receive an AI-generated answer grounded in Austrian legal sources (RIS, LINDE, FINDOK, MANZ, and more). Responses stream over Server-Sent Events (SSE).

Base URL

```
https://app.alawyer.ai
```

Authentication

All requests must include your API key as a bearer token:

```
Authorization: Bearer YOUR_API_KEY
```

Keep your API key secret. Never expose it in client-side code or commit it to source control.

Rate limits

Requests are limited to **10 per minute** per API key, enforced as a sliding window.

When you exceed the limit, the API responds with `429 Too Many Requests`. Honor the `Retry-After` header before retrying.

Response header	Description
`X-RateLimit-Limit`	Maximum requests allowed in the window.
`X-RateLimit-Remaining`	Requests remaining in the current window.
`X-RateLimit-Reset`	Unix timestamp (seconds) when the window resets (on `429` only).
`Retry-After`	Seconds to wait before retrying (on `429` only).

Create a completion

```
POST /api/v1/completions
```

Submits a legal question and returns the AI-generated answer with cited sources. Responses are delivered over Server-Sent Events (SSE).

Request headers

Header	Value	Required
`Authorization`	`Bearer YOUR_API_KEY`	Yes
`Content-Type`	`application/json`	Yes
`Accept`	`text/event-stream`	Optional, recommended

Request body

Field	Type	Required	Description
`query`	string	Yes	The legal question to answer.
`mode`	string	No	Research depth — one of `base`, `pro`, `max`. Defaults to `pro`.
`response_format`	object	No	Return a structured JSON object instead of prose. See Structured JSON output.

```
{
  "query": "Welche Bedeutung hat die Kontaminierung eines Grundstücks für die Mang
  "mode": "max"
}
```

Response

The connection stays open while Alawyer researches your question. The server first sends a `: connected` comment, then SSE keep-alive comment lines (`: keepalive`) every 15 seconds — both are SSE comments you can safely ignore.

When processing completes, the server sends **one** `data:` event containing the full response, followed by a terminating `data: [DONE]` event:

```
: connected

: keepalive

: keepalive

data: {"id":"resp-1706123456789","response":"Die Kontaminierung eines Grundstücks

data: [DONE]
```

Response payload

Field	Type	Description
`id`	string	Unique identifier for the response.
`response`	string	The AI-generated answer. Inline citations appear as `[N]` markers; each `N` matches a source's `sourceReferenceId`. In `json_schema` mode, this is a JSON-encoded string — parse it again to get your object (see Structured JSON output).
`sources`	array	Legal sources cited in the answer. See Source object.
`created`	integer	Unix timestamp (seconds) when the response was generated.
`processing_time_ms`	integer	Total server-side processing time in milliseconds.
`usage`	object	Token usage for the request. May be omitted.
`usage.prompt_tokens`	integer	Tokens consumed by the prompt.
`usage.completion_tokens`	integer	Tokens generated in the response.
`usage.total_tokens`	integer	Sum of prompt and completion tokens.

Source object

Each item in `sources` describes a legal document the answer is grounded in. Fields vary by source type — all are optional except where your application requires them.

Field	Type	Description
`sourceTitle`	string	Human-readable title of the document.
`sourceType`	string	Source provider — e.g. `RIS`, `LINDE`, `FINDOK`, `MANZ`.
`sourceUrl`	string	URL to the original document.
`sourceChunk`	string	Excerpt used to ground the answer. Wrapped in a `<source>...</source>` XML structure containing `<title>`, `<content>`, and source-type-specific metadata.
`sourceSection`	string	Section identifier within the source (e.g. `§ 284` for a statute paragraph, or a breadcrumb path for commentary).
`sourceSectionTitle`	string	Title of the cited section.
`sourceDate`	string	Date of the document.
`citation`	string	Formatted citation string.
`rzNumber`	string	Legal reference number (Randziffer), where applicable.
`sourceChunkPageNo`	string	Page number of the cited excerpt.
`sourceReferenceId`	string	Citation index. `[N]` in the `response` text corresponds to the source whose `sourceReferenceId` is `"N"`.

Response time

A single completion typically takes between **1 and 10 minutes**, depending on the complexity of the question and the amount of research required.

Structured JSON output

By default `response` is plain text (Markdown). To receive a machine-parseable object instead, pass a `response_format` with a JSON Schema. Alawyer constrains the model's output to your schema using native structured outputs.

`response_format`

Field	Type	Required	Description
`type`	string	Yes	`"text"` (default, prose) or `"json_schema"`.
`json_schema`	object	When `type` is `"json_schema"`	The schema wrapper (below).

`json_schema`:

Field	Type	Required	Description
`name`	string	Yes	A name for the schema.
`schema`	object	Yes	A JSON Schema. Its root `type` must be `object`.
`description`	string	No	
`strict`	boolean	No	

How your schema is filled

The full answer is written into one **free-form string** field of your schema:

- Exactly **one** string property → the answer goes there.
- **Multiple** string properties → name the one that holds the answer `"answer"`, or the request is rejected with `400`.
- Non-string properties (numbers, booleans, enums) are populated by the model — e.g. a `confidence` number. Put guidance for them in each property's `description`.

> **Important:** For the most reliable JSON, let `response_format` define the structure and keep your `query` to the question itself — there's no need to ask for JSON or include an example object in the prompt. Put any field-specific guidance in the schema's `description` fields.

Response shape

In `json_schema` mode, **`response` is a JSON-encoded string** that you parse a second time to get your object.

Request:

```
{
  "query": "Ist der Ablauf eines befristeten Arbeitsverhältnisses durch die Einber
  "mode": "max",
  "response_format": {
    "type": "json_schema",
    "json_schema": {
      "name": "legal_answer",
      "schema": {
        "type": "object",
        "properties": {
          "answer": { "type": "string" },
          "confidence": {
            "type": "number",
            "description": "Confidence in the answer, between 0 and 1."
          }
        },
        "required": ["answer", "confidence"],
        "additionalProperties": false
      }
    }
  }
}
```

Response — the single `data:` event; note `response` is a string containing JSON:

```
data: {"id":"resp-1706123456789","response":"{\"answer\":\"Ja. Gemäß § 922 ABGB …
```

Parse `response` again to get your object:

```
const payload = JSON.parse(dataLine.slice(6));
if (payload.error) throw new Error(payload.error.message);

const result = JSON.parse(payload.response); // { answer, confidence }
console.log(result.answer, result.confidence);
console.log(`Cited ${payload.sources.length} sources`);
```

Errors

Errors are returned as JSON with the following shape:

```
{
  "error": {
    "message": "Invalid API key"
  }
}
```

For most failure modes the HTTP status reflects the error. However, because headers are sent before processing completes, upstream failures during streaming are returned with HTTP `200` and an `error` field in the response body instead of `response`. **Always check for `error` in the payload before reading `response`.**

Status	Meaning
`200`	Success, or upstream failure surfaced via `error` in the response body.
`400`	Invalid request — e.g. `query` missing or not a string, `mode` not one of `base` / `pro` / `max`, or an invalid `response_format` / schema.
`401`	Missing or invalid API key.
`429`	Rate limit exceeded. Wait `Retry-After` seconds before retrying.
`500`	Internal server error.

JavaScript example

```
const res = await fetch("https://app.alawyer.ai/api/v1/completions", {
  method: "POST",
  headers: {
    Authorization: `Bearer ${process.env.ALAWYER_API_KEY}`,
    "Content-Type": "application/json",
    Accept: "text/event-stream",
  },
  body: JSON.stringify({
    query:
      "Welche Bedeutung hat die Kontaminierung eines Grundstücks für die Mangelhaf
    mode: "max",
    response_format: {
      type: "json_schema",
      json_schema: {
        name: "legal_answer",
        schema: {
          type: "object",
          properties: {
            answer: { type: "string" },
            confidence: {
              type: "number",
              description: "Confidence in the answer, between 0 and 1.",
            },
          },
          required: ["answer", "confidence"],
          additionalProperties: false,
        },
      },
    },
  }),
});

if (res.status === 429) {
  const retryAfter = parseInt(res.headers.get("retry-after") ?? "60", 10);
  throw new Error(`Rate limited. Retry after ${retryAfter}s.`);
}

if (!res.ok) {
  const { error } = await res.json();
  throw new Error(error?.message ?? `Request failed: ${res.status}`);
}

// The SSE response ends with a single `data: { ... }` event containing the
// full payload. Read the body and pick out that line.
const body = await res.text();
const dataLine = body.split("\n").find((line) => line.startsWith("data: {"));
if (!dataLine) throw new Error("No data event received");
```

```
const payload = JSON.parse(dataLine.slice(6));

// Upstream failures arrive as HTTP 200 with an `error` field instead of `response
// Check `error` before reading `response`.
if (payload.error) throw new Error(payload.error.message);

// In json_schema mode, `response` is a JSON string — parse it to get your object.
const result = JSON.parse(payload.response); // { answer, confidence }
console.log(result.answer, result.confidence);
console.log(`Cited ${payload.sources.length} sources`);
```